

8h às 10h

Autoestudos

1

O que é AutoML?

2

O que é Pycaret?

3

Prática: Pycaret para Classificação e Regressão

4

Prática: Pycaret para Classificação Multiclasse

| Stand-up daily: em pé, direto ao ponto

15 min

10:00 → 10:15

1

Em pé

A daily é rápida — todos de pé.

2

Tasks do projeto

O que cada um fez e o que falta.

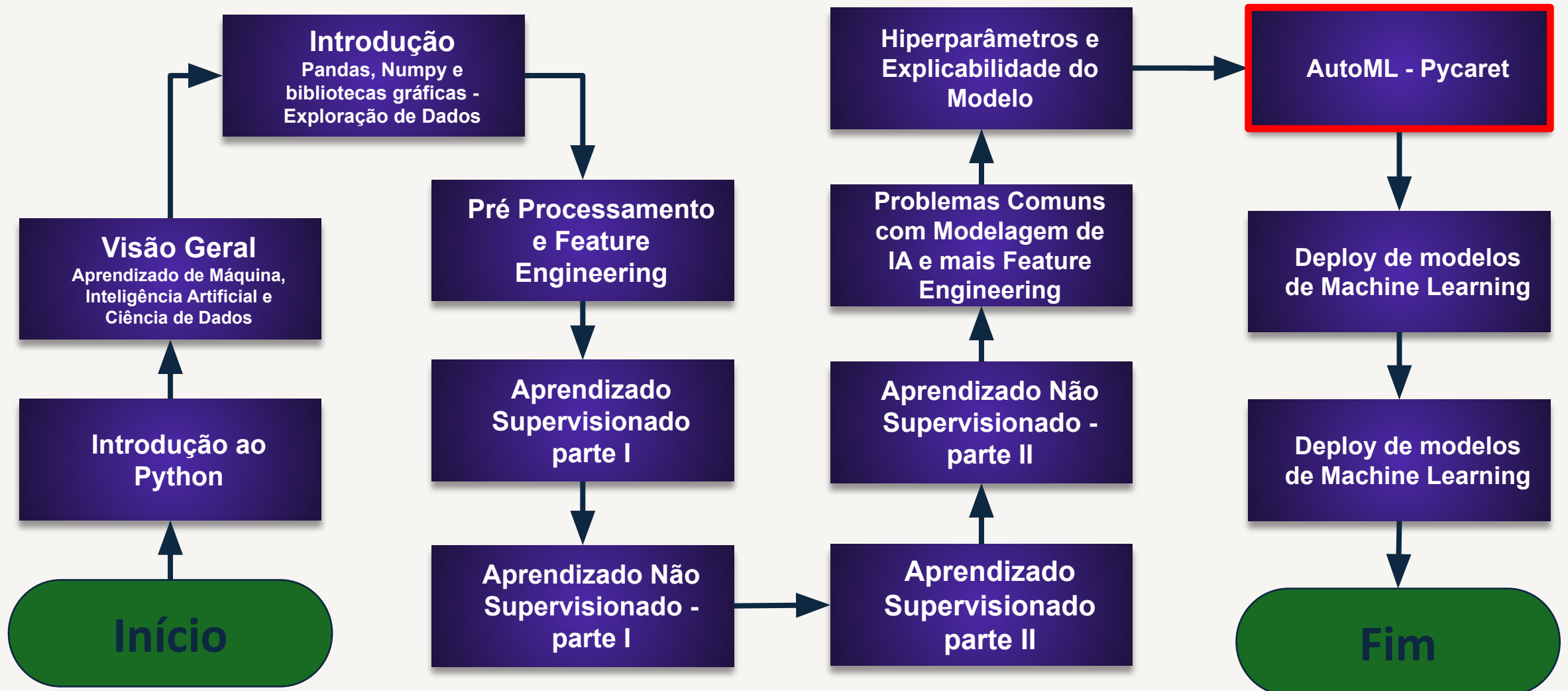
3

Cronograma do dia

Definir prioridades e metas de hoje.

Objetivo: alinhar o time e destravar o dia.

DIAGRAMA DO MÓDULO



AULA 11 • SEMANA 8

AutoML - Pycaret

Setup • Compare Models • Classificação • Regressão •
Classificação Multiclasse

Prof^a Dra. Crishna Irion

Agenda do Encontro

1

Conceito

O que é AutoML e por que usar

Pycaret:

Onde entra no fluxo: definição do problema

AutoML acelera, mas não decide sozinho

Automation bias: cuidado ao confiar demais na ferramenta

2

Aplicação prática

Classificação, Regressão e Classificação

Multiclasse com Pycaret

3

Código

setup() — preparar os dados

compare_models() — comparar algoritmos

tune_model() — ajustar hiperparâmetros

predict_model() — gerar previsões

PARTE 1

O que é AutoML?

Entender o conceito de AutoML, suas vantagens e seus limites.

AutoML não decide por você — ele acelera a exploração.

```
AutoML testado: 15 modelos  
Comparacao pronta em 3 minutos  
Mas o contexto ainda nao foi definido  
O problema esta resolvido?
```

Compara métricas, não o contexto.

Ele não sabe qual é o custo do erro nem se a métrica escolhida é a certa.

Cabe ao cientista de dados decidir o que realmente importa.

Automatiza pré-processamento, seleção de modelo, comparação de métricas e ajuste de hiperparâmetros.

Testa várias famílias de modelos em minutos — mas não substitui a compreensão do problema nem o julgamento crítico dos resultados.

Vantagens x Limites do AutoML

VANTAGENS (o que ganha)

- prototipagem rápida: baseline robusto em pouco tempo
- compara dezenas de algoritmos automaticamente
- democratiza o acesso a ML para quem está começando
- libera tempo para o que exige julgamento humano

LIMITES (o que perde)

- não faz feature engineering criativo nem entende o domínio
- pode mascarar overfitting sem validação cuidadosa
- exige interpretação humana antes de qualquer decisão
- não substitui o entendimento do problema de negócio

**Não existe ferramenta perfeita nos dois lados
— o equilíbrio depende de como você a usa.**

Onde o AutoML entra no fluxo do projeto

1

“Esse problema é tratável?”

os dados que tenho conseguem sustentar um modelo razoável?

2

“Qual família de modelo já performa bem?”

árvore, linear, boosting... sem ajuste fino, o que já se destaca?

3

“Vale a pena investir tempo manual?”

o ganho esperado justifica o esforço de feature engineering e tuning manuais?

Não existe “modelo pronto para produção” só porque o AutoML apontou o melhor — existe um ponto de partida informado.

PARTE 2

O que é Pycaret?

Conhecer a ferramenta, seu fluxo e seus módulos.

| Pycaret em uma frase

Biblioteca open-source de AutoML em Python, de baixo código.

Automatiza pré-processamento, comparação de modelos, tuning, ensembling, interpretação e deploy.

Módulos principais: classification, regression, clustering, anomaly detection, nlp, entre outros.

Por que usar - poucas linhas de código substituem dezenas de linhas de scikit-learn escritas à mão.

O Pycaret é construído sobre bibliotecas que vocês já conhecem — scikit-learn, XGBoost, LightGBM — e apenas orchestra o pipeline em torno delas.

Fluxo típico do Pycaret



1. Setup e comparação

setup() prepara os dados; compare_models() testa e ordena os algoritmos.

```
from pycaret.classification import *  
s = setup(data=df, target='coluna_alvo')  
best_model = compare_models()  
# compara dezenas de algoritmos
```



2. Ajuste e predição

tune_model() refina hiperparâmetros;
predict_model() gera as predições finais.

```
tuned_model = tune_model(best_model)  
final_model = finalize_model(tuned_model)  
predict_model(final_model)  
# gera predições em dados novos
```

setup(), compare_models(), tune_model() e predict_model() automatizam todo o pipeline.

O Pycaret testa tudo o que você pediu — mas não decide por você.

- `setup()` + `compare_models()` rodado
- algoritmos comparados automaticamente

Mas qual métrica importa?

O melhor do ranking é o certo para o problema?

Cabe a você pedir as coisas certas.

O Pycaret não entende o significado das variáveis nem o contexto do negócio.

Não decide sozinho qual métrica importa (precisão x recall, por exemplo).

Não garante que o melhor modelo da comparação seja o certo para produção.

Automation bias: cuidado com a tendência de confiar demais em resultados só por virem de um sistema automatizado.

PARTE 3

Prática: Pycaret para Classificação

Aplicar o fluxo do Pycaret em um problema de classificação binária.

Código: classificação binária



1. Treinar e comparar

setup() prepara os dados; compare_models() testa vários algoritmos.

```
from pycaret.classification import *
clf1 = setup(data=df, target='target')
best_model = compare_models()
# validação cruzada já embutida
```



2. Ajustar e avaliar

tune_model() refina hiperparâmetros; evaluate_model() abre o painel de avaliação.

```
tuned_model = tune_model(best_model)
predict_model(tuned_model)
evaluate_model(tuned_model)
plot_model(tuned_model, plot='confusion_matrix')
# matriz de confusao, ROC, importancia de variaveis
```

Tutorial de referência: Binary Classification — repositório oficial do Pycaret no GitHub.

Além dos gráficos nativos, dá para conectar com o SHAP da Aula 10.

```
interpret_model(tuned_model)
- roda SHAP por baixo dos panos
- mostra contribuicao por variavel
```

Nativa basta ou preciso de mais?

Nativa é rápida; SHAP é fino.

`plot_model(..., plot='feature')` usa a importância nativa do modelo (ex.: árvores).

SHAP explica a contribuição de cada variável em cada previsão individual.

`evaluate_model()` abre um painel interativo com matriz de confusão, curva ROC e importância de variáveis.

Sem escrever código de plotagem manual — tudo já vem pronto no Pycaret.

PARTE 4

Prática: Pycaret para Regressão

Aplicar o mesmo fluxo do Pycaret em um problema de regressão.



Código: regressão



1. Treinar e comparar

`setup()` e `compare_models()` funcionam exatamente como em classificação.

```
from pycaret.regression import *
reg1 = setup(data=df, target='preco')
best_model = compare_models()
# ordenado por MAE, RMSE, R2...
```



2. Métricas de regressão

MAE, MSE, RMSE e R^2 ordenam o ranking — RMSE penaliza mais os erros grandes.

```
tuned_model = tune_model(best_model)
predict_model(tuned_model)
finalize_model(tuned_model)
# métrica "melhor" precisa validar o objetivo real
```

O fluxo é idêntico ao de classificação — só muda o módulo importado.

PARTE 6

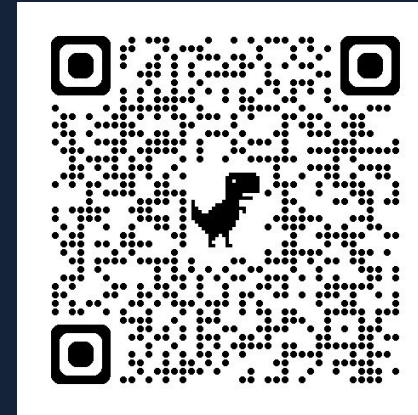
Mão na Massa

Dinâmica offline: Pycaret Humano — um jogo de cartas sobre escolher modelos.

PARTE 6

Mão na Massa

Código de teste. Vamos testar?



<https://encurtador.com.br/fQIF>

Desafio: Aplicar ao projeto AutoML para comparar os resultados

Dúvidas? Vamos conversar.

Profª Dra. Crishna Irion

crishna.irion@prof.inteli.edu.br

